

Mini OS: A Freestanding Operating System Simulation

Implementing Core OS Subsystems in C99 Without Standard Libraries

Vaiditya Tanwar

Newton School of Technology

Rishihood University

Enrollment No.: 230086

vaiditya.t23csai@nst.rishihood.edu.in

Abstract—Mini OS is a freestanding operating system simulation written entirely in C99, with no dependency on the C standard library for core logic. The project implements, from first principles, all major OS subsystems: a first-fit free-list heap allocator with forward coalescing, a custom string library using raw pointer arithmetic, an inode-based virtual filesystem, a preemptive task scheduler driven by timer interrupts, a shell script engine supporting foreground and background execution, and a double-buffered ANSI terminal renderer. The system runs as an interactive Unix process and provides a command-line shell as the primary user interface. Every design decision maps directly to a real operating system concept, making this project both a functional system and an educational model of OS internals. The implementation enforces strict module isolation, signal-safe interrupt handling, and unified memory management across all subsystems. Unit tests cover all core libraries. This paper describes the architecture, methodology, and design rationale of each subsystem, and identifies directions for future extension.

Index Terms—operating systems, memory management, virtual filesystem, preemptive scheduling, signal handling, systems programming, C99

I. INTRODUCTION

Modern operating systems are complex, layered systems that manage hardware resources and present abstractions to user programs. Understanding how these abstractions are implemented — heaps, filesystems, schedulers, device drivers — is fundamental to systems programming yet often opaque to students who interact with them only through library interfaces.

Mini OS addresses this gap by re-implementing each OS subsystem from scratch in C99, using a static memory array as “physical RAM,” POSIX signals as timer interrupts, for raw device I/O, and a 64-slot inode table as a disk partition. The result is a self-contained, interactive operating system simulation that demonstrates, in runnable code, how real OSes solve the problems of memory allocation, file organization, process scheduling, and secure I/O.

The project was built in two phases. Phase 1 (April 7–14) established the core subsystems: memory allocator, string library, screen and keyboard drivers, math engine, virtual filesystem, shell REPL, and cooperative task scheduler, with a full unit test suite. Phase 2 (April 22–24) upgraded the

cooperative scheduler to a preemptive model using `and` added a shell script engine supporting both synchronous (foreground) and asynchronous (background) script execution.

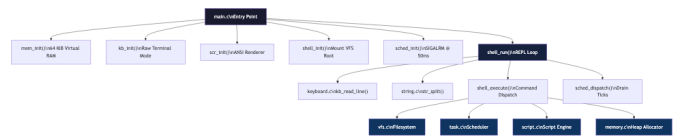


Fig. 1. System architecture and module dependency graph. All heap allocations across every subsystem flow through .

II. METHODOLOGY

A. Memory Management

The heap allocator () implements a **first-fit free list** over a 64 KiB static array declared in :



Each block is prefixed by a struct storing the total block size (including header), a free flag, and a next pointer forming a singly linked list:



Allocation walks the list for the first free block satisfying the requested size. If the block remainder exceeds `min`, it is split into an allocated block and a new free block, minimizing wasted space. All returned pointers are aligned to 8-byte boundaries via the `ALIGN` macro.

Deallocation includes three safety mechanisms: (1) a bounds check rejecting pointers outside the heap region, (2) a double-free guard that silently ignores attempts to free an already-free block, and (3) **forward coalescing** — after marking a block free, the entire list is scanned and all adjacent free blocks are merged. Coalescing prevents fragmentation from

accumulating across many alloc/free cycles and is verified by the stress test in (1,000 allocation/free cycles with full heap recovery).

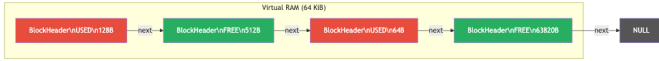


Fig. 2. Heap free list after several allocations. Red = USED, Green = FREE. Forward coalescing merges adjacent green blocks on every .

Every subsystem — VFS file data, task state structs, script line buffers — allocates exclusively through `alloc` and `free`. This unified allocation model allows `memview` to produce a complete, accurate picture of system memory at any moment.

B. File System

The virtual filesystem (`VFS`) is modelled directly on Unix inode semantics. A 64-slot `inode` stores all file and directory metadata; file content is heap-allocated via `alloc`.



A `dir` tracks total file and directory counts and the current working directory inode index. The directory tree is represented implicitly through the `parent` field: to list a directory, all inodes with a matching `parent` index are enumerated. This mirrors how real on-disk filesystems organize entries.

Supported operations include `open`, `close`, `lstat`, `stat`, and `traverse` (including `traverse` and `traverse`). The `open` operation verifies that directories are empty before deletion and frees file data back to the heap allocator, preventing memory leaks. The `stat` function exposes raw file content for consumption by the script engine without going through the terminal output path.

C. Process Management

1) *Task State Machine*: Tasks are represented as entries in an 8-slot table. Each entry holds a function pointer (`fn`), a heap-allocated state pointer, and a `enum`:

State	Behaviour
	is called on every scheduler dispatch
	decrements; transitions to READY at 0
	Slot is reaped (freed) automatically on next pass

2) *Preemptive Scheduler*: Phase 2 upgraded the scheduler from a cooperative model (manual `yield` calls) to a preemptive model using `sigalrm`. The design maps directly to hardware timer interrupt handling in a real kernel:

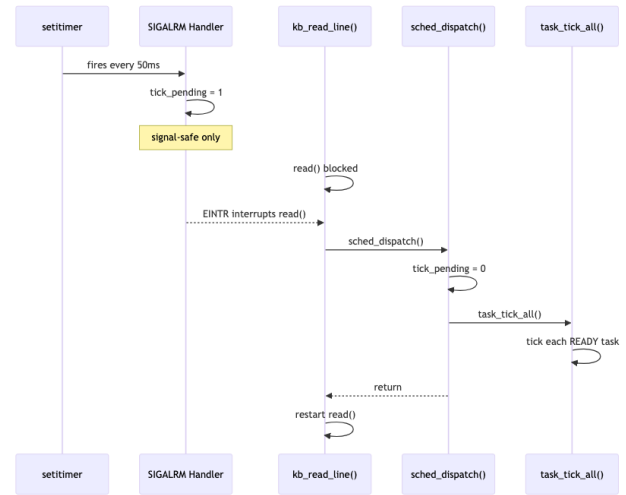


Fig. 3. Preemptive scheduler sequence. The signal handler (ISR) sets only a flag. `read` performs the real work in safe context.

`arms`. The signal handler sets a `flag` and returns immediately — no `yield`, no `sleep`, no function calls. Signal-unsafe operations in a handler are undefined behaviour and can corrupt process state; the flag pattern is the POSIX-compliant solution. `clear` clears the flag and calls `run` from normal execution context, mirroring how kernel schedulers run after saving the interrupted process's context.

`handles` (the error returned when a blocking `read` is interrupted by a signal) by calling `restart_read` before restarting the read, ensuring background tasks tick continuously even during user input.

3) *Shell Script Engine*: Scripts are plain VFS files (one command per line; lines are comments). `exec` executes lines synchronously by calling `run` for each — the same dispatch path as interactive commands. `alloc` allocates a `struct` on the heap, loads the line table, and registers a `fn` as a scheduler task. Each 50 ms tick executes one line; when all lines are consumed the task sets `READY` and the slot is reaped automatically.

D. I/O Management

1) *Keyboard Input*: `kb_read_line` uses `termios` to configure the terminal in raw mode (`termios` and `termios` disabled, `termios`). `kb_read_line` switches to blocking mode for the read loop, handles backspace with cursor manipulation, filters control characters, decodes multi-byte arrow key escape sequences (`termios`), and restarts on `termios`. The original terminal configuration is saved and restored via `termios`.

2) *Screen Output*: `screen_output` implements a double-buffered ANSI renderer with diff-based refresh. A logical `buffer` accumulates all drawing calls; `screen_output` compares each cell against the physical `buffer` and emits ANSI cursor-positioning and colour sequences only for changed cells. All output is accumulated in a local write buffer and flushed in a single `write` call, eliminating flicker from multiple small writes. Console wrappers (`write`, `write`) are used throughout all modules for terminal output.

E. Security, Error Handling, and Halt

Mini OS treats correctness and safety as first-class concerns across all subsystems:

Memory safety. performs bounds checking (pointer outside heap rejected), double-free protection (silently ignored), and NULL checking. returns on out-of-memory rather than crashing.

Filesystem safety. All VFS operations validate inputs: zero-length names are rejected, duplicate names in the same directory are rejected, directory removal is refused if the directory is non-empty, and writing to a directory path is refused with an explicit error.

Signal safety. The SIGALRM handler is strictly signal-safe: it performs no I/O, no allocation, and calls no non-reentrant functions. is set so interrupted restartable syscalls are handled by the kernel rather than requiring manual restarts.

String safety. and always bound-check against the supplied and always null-terminate. handles NULL arguments explicitly. Numeric conversions (,) handle edge cases (zero, negatives, non-numeric input, INT_MIN) without undefined behaviour.

Halt. The command sets , causes the REPL loop to terminate, and is called before returns, restoring the terminal to its original state. The registration of ensures restoration even on abnormal termination.

Task safety. validates the task ID range and active flag before freeing state. Background script tasks that exhaust their line table self-transition to rather than requiring external cleanup.

III. RESULTS AND EVALUATION

TABLE I
UNIT TEST RESULTS

Suite	Coverage	Tests
	Alloc, free, split, coalesce, OOM, stress	17
	Arithmetic, spatial, RNG determinism	18
	All string functions incl. tokenizer	28
	Integration: parse → execute pipeline	22
Total		85

All 85 tests pass. The memory stress test runs 1,000 allocation/free cycles across 10 iterations (100 allocations per cycle, variable sizes 32–96 bytes) and verifies that available memory is fully recovered and the heap coalesces to a single block after each cycle. The string tokenizer test exercises leading/trailing delimiters, multiple consecutive delimiters, empty input, and single-token strings. The shell integration test traces a full command from tokenization through output reconstruction.

IV. FUTURE SCOPE

Syscall interface. A formal dispatch table (analogous to a system call table) would cleanly separate “userspace” shell commands from “kernel” subsystem operations, enforcing the privilege boundary that real OSes maintain.

Inter-process communication. Pipes () and message queues between tasks would enable composable commands and demonstrate IPC primitives (shared memory, ring buffers, semaphores).

Command history and tab completion. Storing the last N commands in a ring buffer and completing filenames by scanning VFS inodes would significantly improve shell usability with minimal complexity.

File permissions and timestamps. Adding , , and fields to inodes and surfacing them in output would complete the Unix inode model.

Network simulation. A loopback “network” with / primitives and a virtual socket table would extend the I/O subsystem to cover the network layer.

Bootloader simulation. A stage that loads the OS image from a simulated sector array before would model the BIOS/UEFI handoff to the kernel.

REFERENCES

- [1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ: Wiley, 2018.
- [2] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Upper Saddle River, NJ: Pearson, 2015.
- [3] D. E. Knuth, *The Art of Computer Programming, Vol. 1: Fundamental Algorithms*, 3rd ed. Reading, MA: Addison-Wesley, 1997.
- [4] W. R. Stevens and S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. Upper Saddle River, NJ: Addison-Wesley, 2013.
- [5] IEEE and The Open Group, *The Open Group Base Specifications Issue 7 (POSIX.1-2017)*, IEEE Std 1003.1-2017. [Online]. Available: <https://pubs.opengroup.org/onlinepubs/9699919799/>
- [6] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed. Englewood Cliffs, NJ: Prentice Hall, 1988.
- [7] D. P. Bovet and M. Cesati, *Understanding the Linux Kernel*, 3rd ed. Sebastopol, CA: O’Reilly Media, 2005.